

Reviewer Pack — Minimal Rank of Tropical Curves over Function Fields vs Reso...

Entry 0d128c7b5840 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 16:14:02 UTC

Minimal Rank of Tropical Curves over Function Fields vs Resolution Proof Width

Entry ID: 0d128c7b5840 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 16:14:02 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Tropical Curves) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

[‘For each Boolean function f with n variables, let C_f be the minimal tropical curve representing f in the max-plus semiring. Then, the width of any resolution proof for f is upper-bounded by a function $\theta(n)$ such that $\theta(n) = O(\log^2(n))$.’, ‘Equivalently: For all Boolean functions f with n variables, there exists a constant c_f such that any resolution proof of f has width at most $c_f \log^2(n)$.’]

Rationale (proposer’s reasoning):

[‘The study of tropical curves provides a geometric framework for understanding the structure of polynomials over the max-plus semiring, which can be applied to Boolean functions. This conjecture suggests a potential relationship between the geometric properties of these curves and the complexity of resolution proofs, which is a key proof system in complexity theory.’]

Taxonomy category: TROPICAL_GEOMETRY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 67ed560c036bcd8e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all Boolean functions f with n variables, the resolution proof width is at most a constant multiple of $\log^2(n)$ and not greater than 1.5 times $\log^2(n)$ for any seed. The conjecture is falsified if there exists a function f such that its resolution proof width exceeds 1.5 times $\log^2(n)$ for any seed.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND resolution proof complexity AND minimal rank - minimal tropical curve AND resolution proof width AND function fields - Boolean function AND max-plus semiring AND resolution proof complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_minimal_tropical_curve(f):
        n = len(f)
        if n == 1:
            return f[0]
        else:
            mid = n // 2
            left = generate_random_boolean_function(mid)
            right = generate_random_boolean_function(n - mid)
            curve = [max(left[i], right[i]) for i in range(n)]
            return sum(curve, key=lambda x: -x)

    def compute_resolution_proof_width(f):
        n = len(f)
        if n == 1:
            return 0
        else:
            mid = n // 2
            left = generate_random_boolean_function(mid)
            right = generate_random_boolean_function(n - mid)
            width_left = compute_resolution_proof_width(left)
```

```

        width_right = compute_resolution_proof_width(right)
        return max(width_left, width_right) + 1

def log_squared(n):
    if n <= 0:
        return 0
    return math.log2(n) ** 2

n_values = [5, 10, 15, 20, 30, 40]
widths = []

for n in n_values:
    f = generate_random_boolean_function(n)
    curve = compute_minimal_tropical_curve(f)
    width = compute_resolution_proof_width(f)
    widths.append(width)

mean_width = sum(widths) / len(widths)
max_width = max(widths)
log2_n = math.log2(len(widths))
upper_bound = 1.5 * log2_n ** 2

conjecture_holds = max_width <= upper_bound
counterexample = "" if conjecture_holds else f"max_width={max_width}, upper_bound={upper_bound}"

return {
    "metric_name": "resolution_proof_width",
    "metric_value": mean_width,
    "instances_tested": len(widths),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"max_width_exceeds_upper_bound\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7702f4c2.py", line 84, in <module>
    trial_result = run_trial(seed)
                   ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7702f4c2.py", line 57, in run_trial
    curve = compute_minimal_tropical_curve(f)
           ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7702f4c2.py", line 33, in compute_minimal_tropical_curve
    return sum(curve, key=lambda x: -x)
           ~~~~~~
```

TypeError: 'key' is an invalid keyword argument for sum()

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means we cannot verify the conjecture's conditions. | next: Re-run the test to ensure it completes without crashing and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11183
2	preregistration	ollama_remote	glm4:latest	0	0	5816
3	novelty	ollama_remote	glm4:latest	0	0	4642
4	novelty	ollama_remote	glm4:latest	0	0	5327
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22833
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18723
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10900
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10897
9	judge	ollama_remote	glm4:latest	0	0	43488

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 133809 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0d128c7b5840.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0d128c7b5840.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0d128c7b5840.tar.gz` (if generated)